Semana 5: validação do Hash e recepção do Flow ID

Projeto: CI Digital Inatel, T25S2, Grupo 2. Hash: Samuel. Flow ID: Bruno. Parser: Herman. Período da atividade: 12 a 18 de outubro de 2026. Simulações executadas em 13/10/2026, horário de Brasília (14/10/2026 UTC). Documentação revisada em 15/10/2026.

As atividades da seção 2.4 da Semana 5 foram implementadas e testadas nesta versão de trabalho. O Hash produziu 181 resultados CRC16 iguais aos esperados; o Flow ID publicou 181 registros iguais aos esperados. Foram conferidos os dados completos, a ordem dos registros, `start`, a duração de `valid` e o reset. Este guia explica o processo e reúne as evidências para Bruno e Samuel compartilharem com o grupo.

1. Fixe o escopo e o contrato que esta versão utiliza.

O anexo `Semana 5.docx` pede a simulação do Hash com valores conhecidos, a implementação da recepção do Flow ID e evidências das duas simulações. A seção 3 determina clock comum, protocolo sem `ready` e recepção com captura dos dados e geração de `start` para iniciar a máquina de estados, ou FSM.

A base da implementação foi o commit `448214edd429c9655c87580b86c92a992d842a69`. Na revisão de 15/10, o GitHub estava em `7388f64d54f2a8f3f20f727e7340258f6dba01ea`: os dois ZIPs de Hash e Flow ID permanecem iguais; as mudanças são na PoC do Parser. Os RTLs desses ZIPs ainda contêm `ready`, exigindo a adaptação para o contrato atual. As versões da Semana 4 foram preservadas; este pacote contém as versões de trabalho da Semana 5.

Item	Versão publicada da Semana 4	Versão de trabalho da Semana 5
Hash	Entrada e saída com `ready`	`in_valid` e `out_valid`; pulso de saída de um ciclo
Flow ID	Saída com `out_ready`	Recepção registrada, `start` interno e FSM
CRC	CCITT-FALSE, 16 bits	Mesma função e mesma ordem de bytes
Dados	Chave 104, comprimento 16, timestamp 80	Larguras preservadas
Armazenamento	Sem FIFO	Sem FIFO; um registro pendente por módulo
Reset	`reset_n`, síncrono, ativo baixo	Convenção preservada nos dois RTLs

O trecho de referência da Semana 5 utiliza `rst` ativo alto e assíncrono. Os RTLs publicados utilizam `reset_n` ativo baixo e síncrono. Esta versão aplica o padrão de recepção mantendo o reset existente. **A conexão de reset do sistema ainda precisa ser uniformizada entre as equipes**; as evidências abaixo se referem explicitamente a `reset_n` síncrono. O exemplo do enunciado não foi tratado como uma troca silenciosa da porta de reset.

Os sinais `start` e `capture_allowed` são internos. `capture_allowed` protege o registro ocupado; não existe retorno de disponibilidade para o emissor. Cada publicação usa `valid` alto por um ciclo e dados estáveis na borda de subida. Com `valid` baixo, o receptor ignora o barramento.

Origem e destino	Porta de origem	Porta de destino	Bits
Parser -> Flow ID	src_ip	src_ip_in	32
Parser -> Flow ID	dst_ip	dst_ip_in	32
Parser -> Flow ID	src_port	src_port_in	16
Parser -> Flow ID	dst_port	dst_port_in	16
Parser -> Flow ID	protocol	protocol_in	8
Parser -> Flow ID	packet_length	packet_length_in	16
Parser -> Flow ID	timestamp	timestamp_in	80
Parser -> Flow ID	metadata_valid	in_valid	1
Flow ID -> Hash	flow_key_out	flow_key_in	104
Flow ID -> Hash	packet_length_out	packet_length_in	16
Flow ID -> Hash	timestamp_out	timestamp_in	80
Flow ID -> Hash	out_valid	in_valid	1
Hash -> Controller	hash_index_out	hash_index	16
Hash -> Controller	flow_key_out	flow_key	104
Hash -> Controller	packet_length_out	packet_length	16
Hash -> Controller	timestamp_out	timestamp	80
Hash -> Controller	out_valid	hash_valid	1

Ambos os módulos também recebem `clk` e `reset_n`, de um bit cada. O Parser atualizado já usa 80 bits, corrigindo a largura anterior de 64. Seu conversor ainda fornece um contador linear de nanossegundos; a seção 2.1 da Semana 5 pede segundos/nanossegundos em campos separados. Por exemplo, 1,234567 s corresponde a `0000000000004995FF58` no contador linear e a `0000000000010DFB3558` no formato acordado. O ambiente de verificação deve alinhar essa representação. Flow ID e Hash preservam os 80 bits recebidos. A palavra de 32 bits da Flow Table mantém o índice do Hash em 16 bits.

2. Defina exatamente quais bytes entram no CRC.

O Flow ID concatena os cinco campos nesta ordem:

```
flow_key = {src_ip, dst_ip, src_port, dst_port, protocol};
```

Campo	Bits	Posição na chave
IP de origem	32	flow_key[103:72]
IP de destino	32	flow_key[71:40]
Porta de origem	16	flow_key[39:24]
Porta de destino	16	flow_key[23:8]
Protocolo	8	flow_key[7:0]

A chave de 104 bits contém 13 bytes. O primeiro é `flow_key[103:96]`; o último é `flow_key[7:0]`. O Hash processa os campos na ordem da tabela, com o byte mais significativo primeiro. A chave identifica um fluxo unidirecional: inverter origem e destino produz outra chave.

O perfil adotado é CRC-16/CCITT-FALSE, também catalogado como CRC-16/IBM-3740. O nome CRC16, sozinho, é insuficiente para definir os resultados: os parâmetros também precisam coincidir.

Parâmetro	Valor utilizado
Largura	16 bits
Polinômio	`0x1021`
Inicialização	`0xFFFF`
Reflexão de entrada e saída	Desativadas
XOR final	`0x0000`
Sequência processada no DUT	13 bytes da chave, MSB primeiro
Saída	Todos os 16 bits do CRC

Para cada byte, o RTL faz XOR entre o CRC corrente e o byte colocado nos oito bits superiores. Em seguida, realiza oito atualizações: desloca o CRC à esquerda e aplica XOR com `0x1021` quando o bit superior anterior era 1. O laço de oito iterações descreve lógica combinacional dentro de um clock; esta arquitetura processa um byte por clock.

Comprimento e timestamp são associados ao mesmo pacote e seguem sem alteração. O timestamp é transportado como 80 bits; conforme a Semana 5, `timestamp[79:32]` representa segundos de 48 bits e `timestamp[31:0]` representa nanossegundos de 32 bits. Esses campos temporais não entram no CRC. A Hash Unit também não calcula endereço físico da memória nem decide HIT, MISS ou COLLISION.

3. Prepare os resultados esperados antes de executar o RTL.

O arquivo `python/reference_model.py` usa `binascii.crc_hqx(key, 0xFFFF)`, da biblioteca padrão Python. Essa implementação é independente da função SystemVerilog. O gerador fixa referências conhecidas e verifica o modelo antes de produzir `tb/vectors.mem` e `tb/vectors_expected.csv`.

```
import binascii

assert binascii.crc_hqx(b"123456789", 0xFFFF) == 0x29B1
key = bytes.fromhex("C0A8010AC0A80114C35001BB06")
assert len(key) == 13
assert binascii.crc_hqx(key, 0xFFFF) == 0xB511
```

A sequência ASCII `123456789` contém nove bytes e confirma o perfil do modelo. A porta do DUT tem treze bytes; os testes do módulo utilizam chaves de treze bytes, incluindo a sequência ASCII `123456789ABCD`. O teste de nove bytes não é apresentado como uma transação dessa porta.

A tabela abaixo foi preenchida com o CSV produzido pela simulação. Os valores de CRC são hexadecimais. **A coluna obtido vem de** `hash_index_out` **do RTL**, e o esperado vem do modelo anterior à simulação.

Caso	Chave de 13 bytes, em hexadecimal	Esperado	Obtido	Resultado
0	C0A8010AC0A80114C35001BB06	B511	B511	PASS
1	C0A8010AC0A80114C35001BB06	B511	B511	PASS
2	C0A80114C0A8010A01BBC35006	EB45	EB45	PASS
3	0A0000010A0000023039003511	890D	890D	PASS
4	0A000020C0000220C00001BB06	83A4	83A4	PASS
5	0A000020C0000221C00001BB06	C604	C604	PASS
6	0A000020C0000222C00001BB06	08E4	08E4	PASS
7	0A000020C0000223C00001BB06	4D44	4D44	PASS
8	C0A8010A08080808CB8401BB06	A018	A018	PASS
9	00000000000000000000000000	280C	280C	PASS
10	FFFFFFFFFFFFFFFFFFFFFFFFFF	FED3	FED3	PASS
11	31323334353637383941424344	36CC	36CC	PASS
12	C0A8010BC0A80114C350B9DA06	B511	B511	PASS

Os casos 0 e 1 têm a mesma chave, mas comprimento e timestamp diferentes: ambos produzem `B511`, preservando seus próprios metadados. O caso 12 tem outra chave e também produz `B511`. Isso demonstra uma colisão de índice e confirma a necessidade de encaminhar a chave completa.

Além desses 13 casos dirigidos, há 104 chaves com um único bit ativo e 64 casos pseudoaleatórios com semente 392. Os timestamps nominais usam segundos/nanossegundos; o padrão de todos os bits em 1 é um teste de preservação de largura, sem representar nanossegundos normalizados. Os testes exercitam o barramento, sem exigir pacotes PCAP semanticamente válidos para cada padrão de bits.

4. Acompanhe a recepção e o cálculo na Hash Unit.

O arquivo `rtl/hash_unit.sv` captura chave, comprimento e timestamp como um registro único de 200 bits. A captura ativa `start`. A FSM lê esse `start` na borda seguinte, inicializa o CRC e processa os treze bytes. Ao concluir, apresenta o índice junto com os mesmos 200 bits recebidos, totalizando 216 bits de saída.

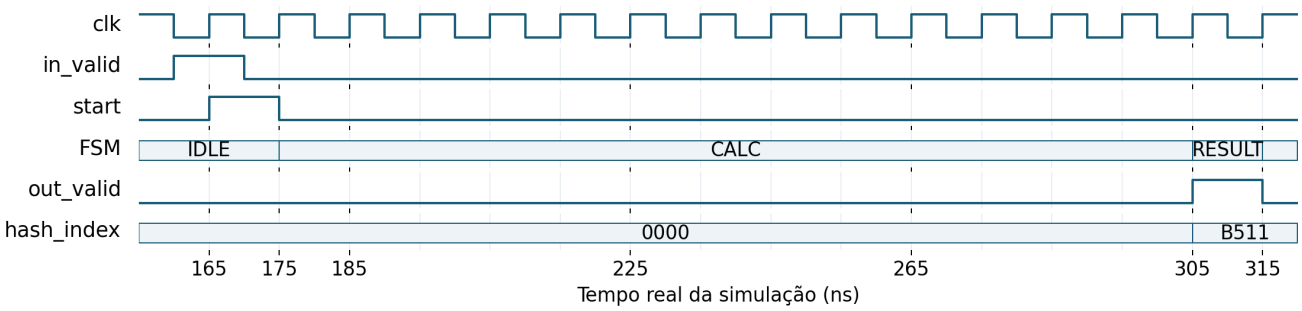
Foram retiradas as portas `in_ready` e `out_ready`. O estado `RESULT` dura um ciclo e gera `out_valid`; depois, a FSM volta a `IDLE`. O último byte usa `next_crc`, que já inclui sua contribuição, para registrar o resultado final.

Considere E0 a borda de captura. A tabela diferencia a disponibilidade depois de uma borda da leitura pelo receptor na borda seguinte.

Borda	Ação	Situação depois da borda
E0	Captura os 200 bits	`start=1`; FSM ainda em `IDLE`
E1	FSM recebe `start`	Entra em `CALC`; CRC inicial `FFFF`
E2 a E13	Processa bytes 0 a 11	Acumula o CRC
E14	Processa byte 12	Entra em `RESULT`; resultado válido
E15	Receptor lê a saída	FSM volta a `IDLE`; `out_valid=0`
E16	Primeira nova captura admissível	Início da próxima transação

Hash: captura, start e processamento dos 13 bytes

Caso 0: captura em 165 ns; leitura da saída em 315 ns. VCD real do DUT isolado.



No caso 0, a captura ocorreu em 165 ns. O resultado `B511` ficou disponível depois de 305 ns, e o monitor o leu na borda de 315 ns: 15 clocks após a captura. O pulso de saída durou um clock, de 305 a 315 ns. A figura foi extraída do VCD real, sem desenhar sinais esperados artificialmente.

5. Acompanhe a lógica de recepção do Flow ID.

O arquivo `rtl/flow_identification_unit.sv` captura os sete campos recebidos do Parser na mesma borda: os cinco campos da chave, o comprimento e o timestamp. Usa registradores internos para manter os 200 bits estáveis. O núcleo da recepção é:

```
start <= 1'b0;
if (in_valid && capture_allowed) begin
    src_ip_reg    <= src_ip_in;
    dst_ip_reg    <= dst_ip_in;
    src_port_reg  <= src_port_in;
    dst_port_reg  <= dst_port_in;
    protocol_reg  <= protocol_in;
    length_reg    <= packet_length_in;
    timestamp_reg <= timestamp_in;
    start         <= 1'b1;
end
```

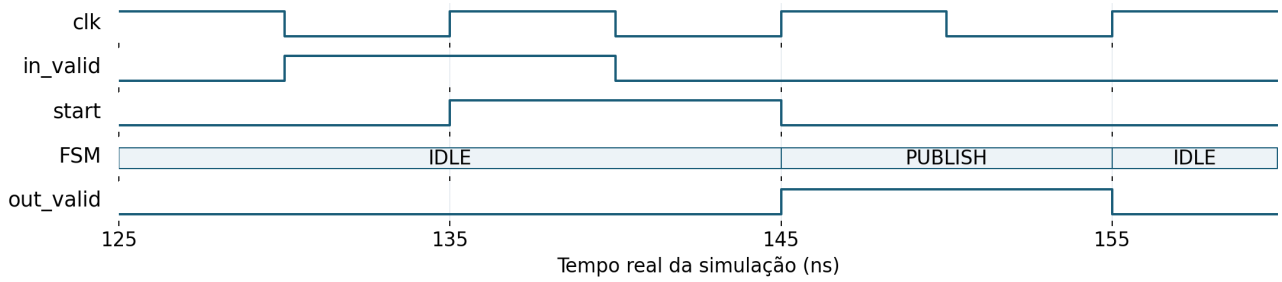
Esse trecho está dentro do bloco com reset do RTL completo. `capture_allowed` significa FSM em `IDLE` e `start` desativado. A condição evita sobrescrever um registro pendente. A ausência de FIFO pressupõe publicações espaçadas; o registro ocupado não constitui uma fila para receber outro pacote.

A FSM tem dois estados: `IDLE` espera `start`; `PUBLISH` encerra a publicação e volta a `IDLE`. Ao observar `start`, monta a chave com os campos já capturados, registra comprimento e timestamp e ativa `out_valid` por um ciclo. Como as atribuições são não bloqueantes, a FSM observa o `start` da borda anterior.

Borda	Ação	Situação depois da borda
E0	Captura os sete campos	`start=1`; sem saída válida
E1	FSM monta e registra a saída	`out_valid=1`; estado `PUBLISH`
E2	Receptor lê os 200 bits	`out_valid=0`; volta a `IDLE`
E3	Primeira nova captura admissível	Início da próxima transação

Flow ID: recepção e publicação do registro

Caso 0: captura em 135 ns; leitura da saída em 155 ns. VCD real do DUT isolado.



No caso 0, o Flow ID capturou em 135 ns, publicou depois de 145 ns e teve sua saída lida em 155 ns. O monitor conferiu a chave completa `C0A8010AC0A80114C35001BB06`, comprimento 60 e timestamp `8000000000010000007B`. Esse timestamp exercita inclusive o bit 79, evitando que uma implementação limitada a 64 bits passe despercebida.

Os limites locais desta implementação são **3 clocks entre capturas do Flow ID** e **16 clocks entre capturas do Hash**. Ao ligar os dois, o emissor deve respeitar pelo menos o limite do Hash; a capacidade do Controller pode exigir intervalo maior. O teste isolado rápido do Flow ID não deve ser conectado diretamente ao Hash com intervalo de 3 clocks. A folga informada pelo professor justifica a ausência de FIFO, mas o intervalo real ainda precisa atender aos módulos do caminho.

Uma chegada enquanto o módulo está ocupado não é aceita pelo registro protegido. Na simulação, uma verificação encerra o teste com `PROTOCOL_VIOLATION`. Ela fica fora da síntese e não cria porta externa nem mecanismo de repetição. Também se deve evitar manter `in_valid` alto por vários clocks: cada borda com validade indica uma publicação.

6. Confira o que foi efetivamente simulado.

Foram compilados e executados dois DUTs isolados, com Verilator 5.49, clock de teste de 10 ns e verificação de assertions. A revisão informada pelo executável é `vUNKNOWN-built20260516-4e853d8`. As compilações finais terminaram sem avisos; a supressão `BLKSEQ` aplica-se às atribuições de controle da testbench.

Medida	Hash	Flow ID
Casos nominais comparados	181	181
Entradas capturadas, incluindo reset	183	183
Saídas lidas e comparadas	181	181
Registros cancelados por reset	2	2
Registros pendentes no fim	0	0
Verificações de `start`	2909	553
Latência captura -> leitura	15 clocks	2 clocks
Intervalo mínimo exercitado	16 clocks	3 clocks
Teste de chegada ocupada	Violação detectada	Violação detectada

A conservação dos registros foi confirmada em cada módulo: 183 capturados = 181 entregues + 2 cancelados por reset. O teste cancela um registro antes de `start` iniciar a FSM e outro em processamento no Hash ou antes da leitura da publicação no Flow ID. Uma validade apresentada durante reset é ignorada, e nenhum registro cancelado reaparece depois.

Verificação	Evidência obtida
CRC correto	181 índices iguais ao modelo independente
Ordem e associação	Cada saída comparada ao registro capturado correspondente
Larguras completas	Comparação de 216 bits no Hash e 200 no Flow ID
Dados inválidos	Barramento alterado com `in_valid=0`, sem corromper a transação
Mesma chave e metadados diferentes	Mesmo CRC, preservando comprimento e timestamp próprios
Pulso de saída	`out_valid` não permanece ativo em duas bordas consecutivas
`start` registrado	Corresponde à captura da borda anterior, respeitando reset
Limite de capacidade	Nova entrada com intervalo de 2 clocks gera diagnóstico

Os 362 resultados observados foram gravados pelas testbenches a partir das saídas dos DUTs. O programa Python faz uma segunda conferência desses CSVs contra os vetores independentes. A coluna obtido não é preenchida copiando a coluna esperado. Foram observados 182 pulsos de `start` por DUT: 181 transações nominais e uma transação cancelada depois de iniciar; o outro cancelamento ocorreu antes dessa observação.

Os testes negativos encerram deliberadamente com código 1 e a mensagem `PROTOCOL_VIOLATION`. O executor confirma que esse diagnóstico ocorreu. Assim, as palavras Fatal/Error nos arquivos `*_negative.log` são a evidência esperada desse teste; os testes nominais terminam em PASS.

```
Hash:    183 capturados; 181 entregues; 2 cancelados; 0 pendentes.
Flow ID: 183 capturados; 181 entregues; 2 cancelados; 0 pendentes.
PASS: 362 resultados RTL comparados.
PASS: 2 violacoes intencionais detectadas.
```

7. Reproduza os testes e consulte os arquivos de evidência.

Extraia `semana5_flow_id_hash.zip` e abra um terminal na pasta `semana5_flow_id_hash`. O teste automático requer Python 3.10 ou superior, Verilator, um compilador C++ e `make`. O modelo de referência e o executor usam somente a biblioteca padrão Python.

```
python3 python/run_tests.py
```

Se o Verilator estiver fora do PATH, informe seu executável:

```
python3 python/run_tests.py --tool /caminho/do/verilator
```

O executor gera os valores esperados, compila cada RTL com sua testbench, executa os testes nominais e negativos e grava logs, CSVs e VCDs. O comando é executado novamente para produzir evidências de uma versão alterada; logs antigos não devem ser apresentados como prova de outro RTL. `validation/summary.json` registra contagens, data UTC e SHA-256 dos arquivos utilizados.

Arquivo ou pasta	Finalidade
`rtl/hash_unit.sv`	Hash atualizado para `valid`, captura e `start`
`rtl/flow_identification_unit.sv`	Recepção dos sete campos e FSM do Flow ID
`tb/tb_hash_unit.sv`	Teste isolado do Hash
`tb/tb_flow_identification_unit.sv`	Teste isolado do Flow ID
`tb/vectors.mem` e `tb/vectors_expected.csv`	Estímulos e valores esperados
`validation/*_cases.csv`	Esperado e obtido, campo por campo
`validation/*_observed_raw.csv`	Palavra completa observada diretamente no DUT
`validation/*_simulation.log`	Saída nominal real do simulador
`validation/*_negative.log`	Diagnóstico da violação deliberada
`validation/*.vcd`	Formas de onda reais para inspeção
`evidencias/*.png`	Figuras extraídas dos VCDs
Guia `.md` e `.pdf`	Explicação e relatório dos resultados

Para recriar as figuras, instale `matplotlib` no seu ambiente Python e execute `python3 python/render_waveforms.py` depois dos testes. Essa dependência é apenas das figuras. Os VCDs também podem ser abertos no GTKWave. Selecione `clk`, `reset_n`, `in_valid`, `dut.start`, `dut.state`, `out_valid` e os campos de dados; no Hash, inclua `hash_index_out` e `dut.byte_index`.

O grupo também pode utilizar ModelSim/Questa. Abaixo está a sequência equivalente para o Hash, executada a partir da raiz do pacote. Esses comandos são orientação de reprodução: **a execução certificada neste relatório foi em Verilator**, sem afirmar uma execução local em ModelSim.

```
python python/reference_model.py
vlib work
vlog -sv rtl/hash_unit.sv tb/tb_hash_unit.sv
vsim -c work.tb_hash_unit -do "run -all; quit -f"
```

Para o Flow ID, compile `rtl/flow_identification_unit.sv` com `tb/tb_flow_identification_unit.sv` e simule `work.tb_flow_identification_unit`. O executor

automático com Verilator continua sendo a forma fornecida de executar também os dois testes negativos e conferir os CSVs.

8. Organize a entrega da Semana 5 e comunique o alcance dos resultados.

Entregável da seção 2.4	Material disponível neste pacote
Evidências de simulação do CRC16	Log PASS, VCD e figura do Hash
Casos com esperado e obtido	181 linhas em `hash_unit_cases.csv`; 13 exemplos neste guia
Recepção do Flow ID simulada	RTL, testbench, 181 comparações, log PASS, VCD e figura

Para anexar no GitHub, o guia fica em `Especificação/Extracao de Metadados/`, o RTL do Hash em `rtl/hash_unit/` e o RTL do Flow ID em `rtl/flow_identification/`. Os testes, modelo e evidências precisam acompanhar os códigos. As ligações e temporizações dos passos 1, 4 e 5 são a atualização de contrato documentada para compartilhar entre Bruno, Samuel, Herman e a equipe do Controller. Este pacote não foi publicado automaticamente no repositório.

A validação cobre **os dois módulos isoladamente**, na versão RTL fornecida. Não foi executada uma ligação Parser -> Flow ID -> Hash -> Controller com PCAP, nem síntese ou teste em FPGA física. O clock de 10 ns é um estímulo da simulação e não comprova frequência máxima de operação após síntese. Verilator é um simulador de dois estados; estas evidências não constituem testes de propagação de X/Z.

Antes da integração do caminho completo, a equipe deve compatibilizar o reset, alinhar o formato segundos/nanossegundos e verificar o espaçamento das publicações, incluindo a ocupação do Controller. A largura de 80 bits já está corrigida no Parser; a formação do timestamp a partir do PCAP cabe à verificação, conforme a seção 2.1. Esses pontos não impedem as três entregas isoladas da seção 2.4, concluídas nesta versão. A seção 2.3 atribui o lookup e as classificações de memória à equipe do Controller.

As fontes abaixo registram o enunciado, a base dos RTLs e a revisão do Parser. Os resultados de simulação pertencem aos RTLs deste pacote, identificados por SHA-256.

- `Semana 5.docx`, fornecido por Samuel: seção 2.4, regras da seção 3 e divisão do timestamp na seção 2.1.
- Decisões da reunião de 13/10/2026, fornecidas por Samuel: larguras, `valid` sem `ready`, captura com `start` e clock comum; orientação anterior do professor sobre ausência de FIFO.
- Repositório no commit analisado.
- PoC do Parser revisada em 15/10: RTL, conversor PCAP e README conferidos; Hash e Flow ID não mudaram entre os snapshots.
- ZIP anterior do Hash, blob `c0f3cf34a8c17b3acc9b74f0c7469a6183b1ff9e`.
- ZIP anterior do Flow ID, blob `034b27f485c3b3f22990d70b980fc6cf41ca7703`.
- Diagrama de interfaces, blob `d3de788bb7d5875b6da34fcc58ccc4c2a58306fb`.
- CRC RevEng, catálogo de 16 bits: parâmetros e alias CCITT-FALSE.
- Documentação oficial de `binascii.crc_hqx`: polinômio e valor inicial configurável.

Os arquivos `validation/summary.json`, os CSVs observados e os logs são as fontes dos resultados numéricos apresentados neste guia.